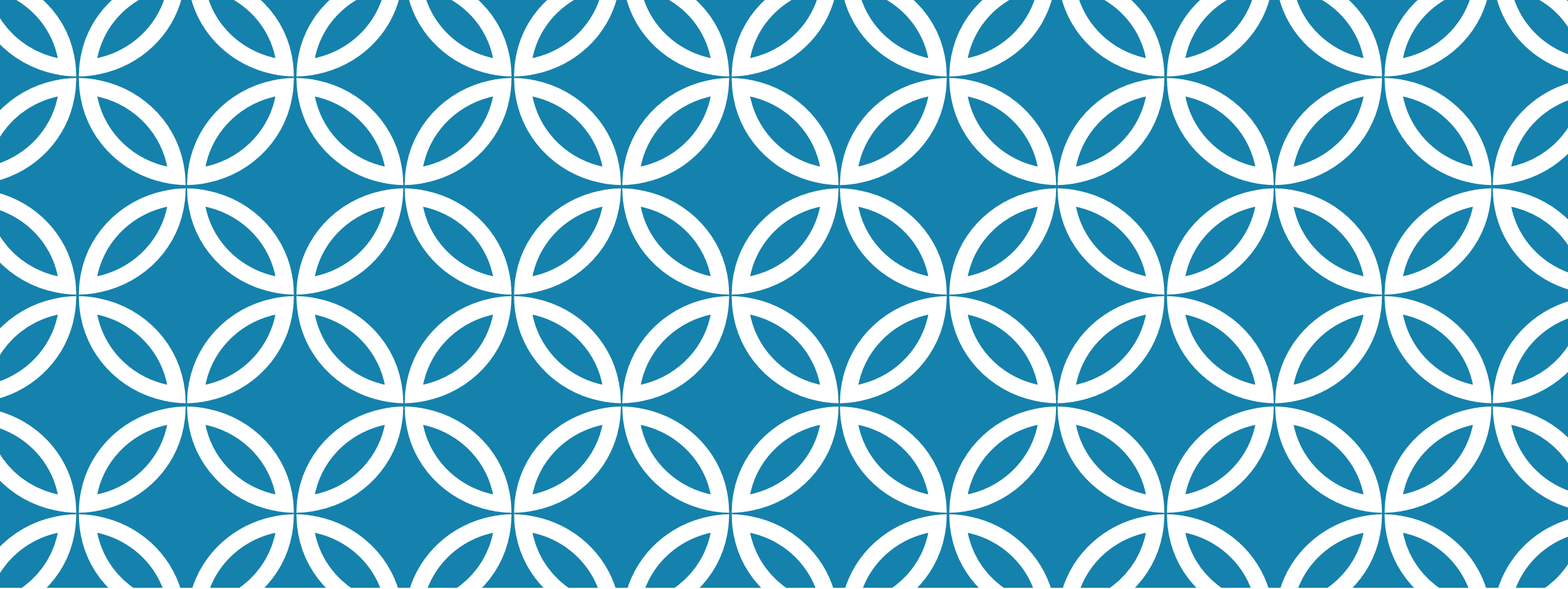




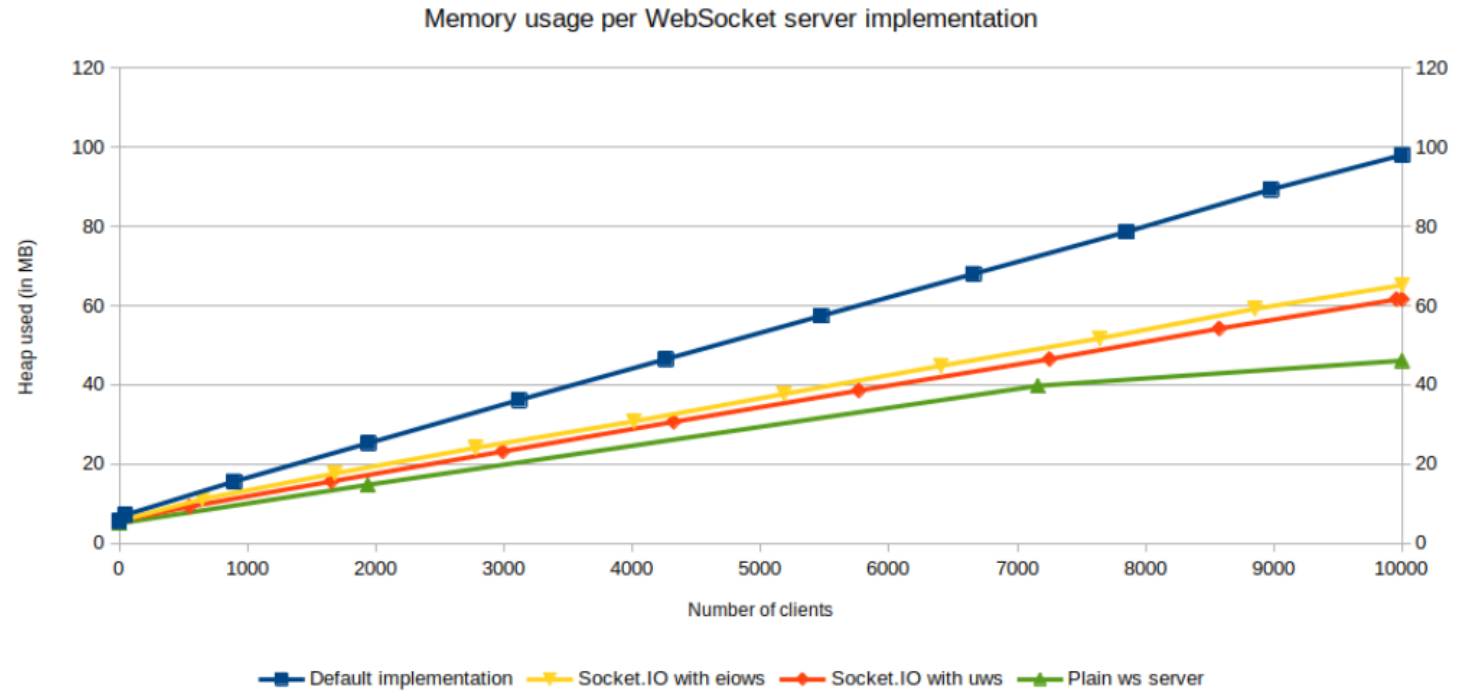
# NODE.JS

Part II

# SOCKET.IO

- ❖ Socket.IO is a library that enables **low-latency, bidirectional** and **event-based** communication between a client and a server.
- ❖ The bidirectional channel between the Socket.IO server (Node.js) and the Socket.IO client (browser) is established with a WebSocket connection whenever possible, and will use HTTP long-polling as fallback.
- ❖ Although Socket.IO indeed uses WebSocket for transport when possible, it adds additional metadata to each packet. That is why a WebSocket client will not be able to successfully connect to a Socket.IO server

# SOCKET.IO



- ❖ Under some particular conditions, the WebSocket connection between the server and the client can be interrupted with both sides being unaware of the broken state of the link.
- ❖ That's why Socket.IO includes a heartbeat mechanism, which periodically checks the status of the connection.
- ❖ And when the client eventually gets disconnected, it automatically reconnects with an exponential back-off delay, in order not to overwhelm the server.

# WRITING A CHAT APPLICATION

- ❑ Create a package.json file to store all the dependency

```
{  
  "name": "socket-chat-example",  
  "version": "0.0.1",  
  "description": "my first socket.io app",  
  "dependencies": {}  
}
```

- ❑ dependencies will be populated automatically in the json file and packages will be installed in a sub directory called node\_modules
- ❑ `npm install express`
- ❑ sample index.js to create a server
- ❑ `npm install -g nodemon`

# SOCKET.IO

- ❑ Express initializes app to be a function handler that you can supply to an HTTP server
- ❑ We define a route handler / that gets called when we hit our website home.
- ❑ We make the http server listen on port 3000.

```
const express = require('express');
const app = express();
const http = require('http');
const server = http.createServer(app);

app.get('/', (req, res) => {
  res.send('<h1>Hello world</h1>');
});

server.listen(3000, () => {
  console.log('listening on *:3000');
});
```

# SOCKET.IO

```
app.get('/', (req, res) => {  
  res.sendFile(__dirname +  
    '/index.html');  
});
```

- ❑ Socket.IO is composed of two parts:
- ❑ A server that integrates with (or mounts on) the Node.JS HTTP Server [socket.io](https://socket.io/)
- ❑ A client library that loads on the browser side [socket.io-client](https://socket.io-client)

```
<html>  
<head>Add CSS</head>  
<body>  
  <ul id="messages"></ul>  
  <form id="form" action="">  
    <input id="input" autocomplete="off"  
    /><button>Send</button>  
  </form>  
</body>  
</html>
```

# SOCKET.IO

```
<script src="/socket.io/socket.io.js"></script>
<script>
var socket = io();
</script>
```

A client library that loads on the browser side [socket.io-client](#)

A new instance of socket.io by passing the server (the HTTP server) object

```
const express = require('express');
const app = express();
const http = require('http');
const server = http.createServer(app);
const { Server } = require("socket.io");
const io = new Server(server);
```

A server that integrates with (or mounts on) the Node.JS HTTP Server [socket.io](#)

❑ To indicate connected users the index.js is updated as follows.

```
io.on('connection', (socket) => {
  console.log('a user connected');
});
```

# CLIENT AND SERVER

```
<script src="/socket.io/socket.io.js"></script>
<script>
var socket = io();
</script>
```

- ❑ index.html should be modified to include the io client
- ❑ If you would like to use the local version of the client-side JS file, you can find it at `node_modules/socket.io/client-dist/socket.io.js`.
- ❑ No URLs are specified when `io()` is called, since it defaults to trying to connect to the host that serves the page
- ❑ loading socket.io client from content delivery network

```
<script src="https://cdn.socket.io/4.5.0/socket.io.min.js"
integrity="..." crossorigin="anonymous"></script>
```

- ❑ events can be added for individual sockets

```
io.on('connection', (socket) => {
  console.log('a user connected');
  socket.on('disconnect', () => {
    console.log('user disconnected');
  });
});
```



## CLIENT SENDS DATA

- ❑ The main idea behind Socket.IO is that you can send and receive any events you want, with any data you want. Any objects that can be encoded as JSON will do, and binary data is supported too.
- ❑ Let's make it so that when the user types in a message, the server gets it as a chat message event

```
<script src="/socket.io/socket.io.js"></script>
<script>
  var socket = io();

  var form = document.getElementById('form');
  var input = document.getElementById('input');

  form.addEventListener('submit', function(e) {
    e.preventDefault();
    if (input.value) {
      socket.emit('chat message', input.value);
      input.value = '';
    }
  });
</script>
```

```
<html>
<head>Add CSS</head>
<body>
  <ul id="messages"></ul>
  <form id="form" action="">
    <input id="input" autocomplete="off"
    /><button>Send</button>
  </form>
</body>
</html>
```

```
io.on('connection', (socket) => {
  socket.on('chat message', (msg) => {
    console.log('message: ' + msg);
  });
});
```

# BROADCAST A MESSAGE

```
io.emit('some event', {Property1: 'value1', Property2: 'value2' });
```

❑ Broadcasts a data to everyone including the sender

❑ If you want to send a message to everyone except for a certain emitting socket, we have the broadcast flag for emitting from that socket:

```
io.on('connection', (socket) => {  
  socket.broadcast.emit('hi');
```

To broadcast the received message, at index.js the following is included

```
io.on('connection', (socket) => {  
  socket.on('chat message', (msg) => {  
    io.emit('chat message', msg);  
  });  
});
```

# RECEIVING MESSAGE FROM OTHERS

on the client side when we capture a chat message event we'll include it in the page

<https://socket.io/get-started/chat>

```
<script>
  var socket = io();

  var messages = document.getElementById('messages');
  var form = document.getElementById('form');
  var input = document.getElementById('input');

  form.addEventListener('submit', function(e) {
    e.preventDefault();
    if (input.value) {
      socket.emit('chat message', input.value);
      input.value = '';
    }
  });

  socket.on('chat message', function(msg) {
    var item = document.createElement('li');
    item.textContent = msg;
    messages.appendChild(item);
    window.scrollTo(0, document.body.scrollHeight);
  });
</script>
```

# ACKNOWLEDGEMENT

## Client

```
socket.timeout(5000).emit('event',  
{ foo: 'bar' }, 'baz', (err,  
response) => {  
  if (err) {  
    // the server did not  
    acknowledge the event in the given  
    delay  
  } else {  
    console.log(response.status);  
    // 'ok'  
  }  
});
```

## Server

```
io.on('connection', (socket) => {  
  socket.on('event', (arg1, arg2,  
callback) => {  
    console.log(arg1); // { foo:  
    'bar' }  
    console.log(arg2); // 'baz'  
    callback({  
      status: 'ok'  
    });  
  });  
});
```

# LISTENING TO ALL

```
socket.on('set users',  
function(msg) {  
    var item =  
document.createElement('li');  
    item.textContent = msg;  
messages.appendChild(item);  
window.scrollTo(0,  
document.body.scrollHeight);  
    });
```

```
socket.onAny((eventName, ...args) =>  
{  
    var item =  
document.createElement('li');  
    item.textContent = args;  
messages.appendChild(item);  
window.scrollTo(0,  
document.body.scrollHeight);  
    });
```

# ROOMS

A *room* is an arbitrary channel that sockets can join and leave. It can be used to broadcast events to a subset of connected clients

```
io.on('connection', (socket) => {  
  // join the room named 'some room'  
  socket.join('BCSEIII group');  
  
  // broadcast to all connected clients in the room  
  io.to('BCSEIII group').emit('Assignments', 'Assignment 3');  
  
  // broadcast to all connected clients except those in the room  
  io.except('BCSEIII group ').emit('hello', 'world');  
  
  // leave the room  
  socket.leave(' BCSEIII group ');  
});
```

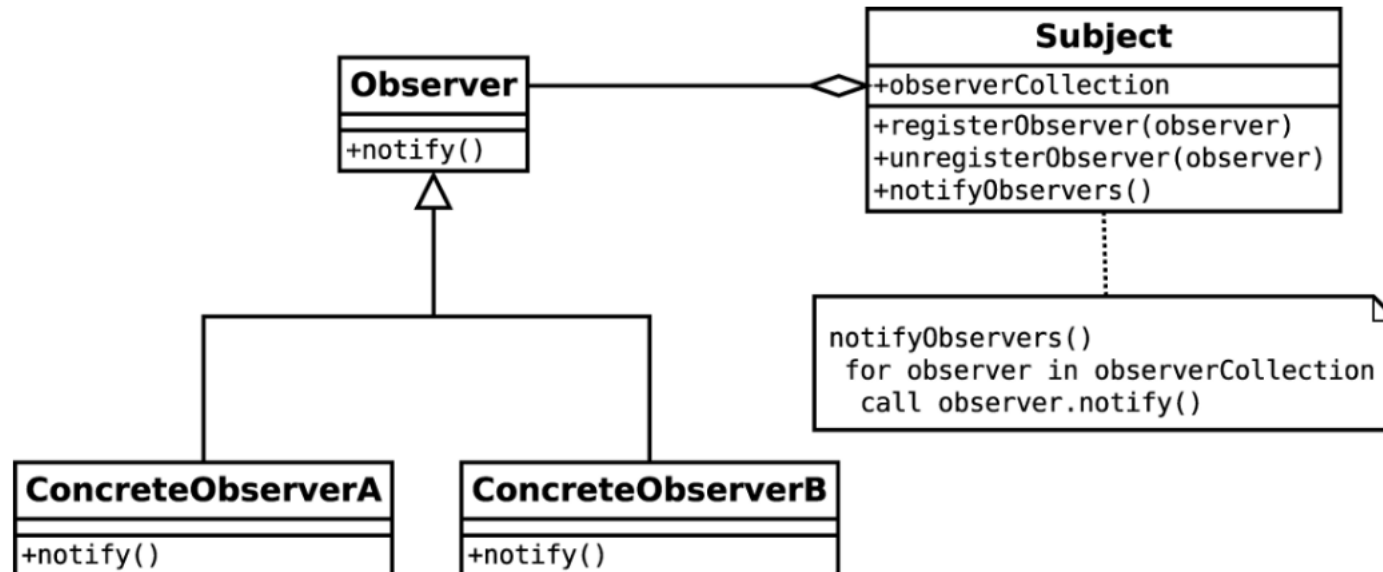
## THE OBSERVER PATTERN

- ❑ Together with reactor, callbacks, and modules, this is one of the pillars of the platform and an absolute prerequisite for using many node-core and userland modules
- ❑ Definition: Pattern (observer): defines an object (called subject), which can notify a set of observers (or listeners), when a change in its state happens.
- ❑ The main difference from the callback pattern is that the subject can actually notify multiple observers, while a traditional continuation-passing style callback will usually propagate its result to only one listener, the callback
- ❑ No need to define interfaces as required in Java to implement this pattern

The observer pattern is already built into the core and is available through the **EventEmitter** class.

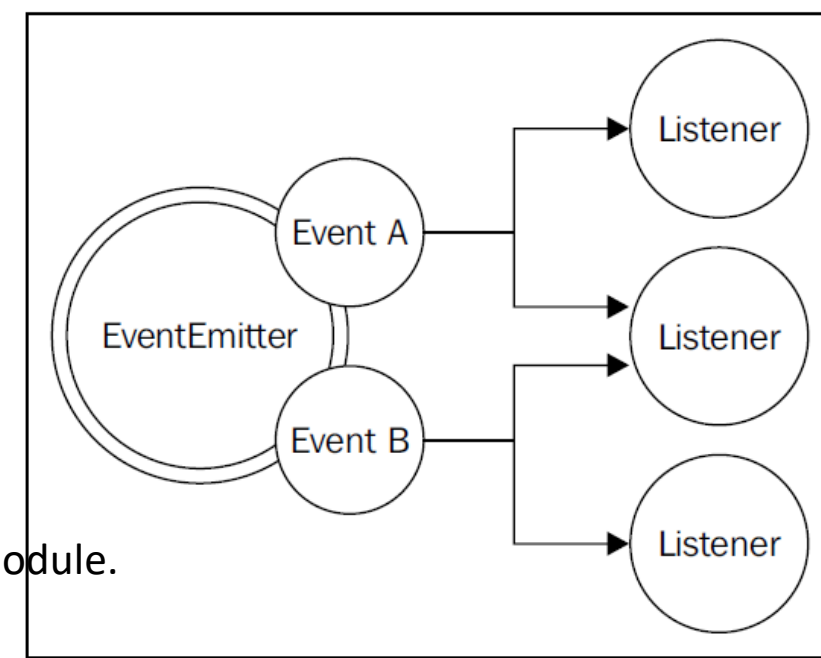
The `EventEmitter` class allows us to register one or more functions as listeners, which will be invoked when a particular event type is fired

# THE OBSERVER PATTERN





# EVENT EMITTER



- ❑ The `EventEmitter` is a prototype, and it is exported from the `events` core module.
- ❑ Obtaining a reference of the `EventEmitter`
  - ❑ `var EventEmitter = require('events').EventEmitter;`
  - ❑ `var eeInstance = new EventEmitter();`
- ❑ The essential methods of the `EventEmitter` are given as follows.
  - `on(event, listener)`: This method allows you to register a new listener (a function) for the given event type (a string)
  - `once(event, listener)`: This method registers a new listener, which is then removed after the event is emitted for the first time
  - `emit(event, [arg1], [...])`: This method produces a new event and provides additional arguments to be passed to the listeners
  - `removeListener(event, listener)`: This method removes a listener for the specified event type

# EVENT EMITTERS

- ❑ All the preceding methods will return the `EventEmitter` instance to allow chaining.
- ❑ There is a big difference between a listener and a traditional Node.js callback; in particular, the first argument is not an error, but it can be any data passed to `emit()` at the moment of its invocation.
- ❑ The `listener` function has the signature, `function([arg1], [...])`, so it simply accepts the arguments provided the moment the event is emitted.
- ❑ Inside the listener, `this` refers to the instance of the `EventEmitter` that produces the event

## EMITTING EVENTS

- ❑ The `EventEmitter` - as it happens for callbacks - cannot just throw exceptions when an error condition occurs, as they would be lost in the event loop if the event is emitted asynchronously.
- ❑ Instead, the convention is to emit a special event, called `error`, and to pass an `Error` object as an argument.
- ❑ A common dilemma when defining an asynchronous API is to check whether to use an `EventEmitter` or simply accept a callback.
- ❑ The general differentiating rule is semantic: callbacks should be used when a result must be returned in an asynchronous way; events should instead be used when there is a need to communicate that something has just happened